

DSC 204: Systems for Scalable Analytics

Programming Assignment 2

May 6, 2026

1 Introduction

This assignment has two parts. In the first part, we will conduct feature engineering for the Amazon dataset. In the second part, we will train ML models using the extracted features. We will use Apache Spark on an DSMLP¹ cluster. You will need to login to the **login node**² of the DSMLP cluster. You will then connect to the cluster and finish all the developments and tests there. You are not expected to code anything locally.

Note: We highly encourage everyone to start early. Go through the setup instructions and task definitions more than once if needed. Try to follow the codebase provided in the **src** directory to gain a better understanding of the assignment, how the data is flowing, how the schemas are being enforced, etc. In real world, you'll most likely be given just the dataset with almost no directions. If you are not familiar with the Spark API and codebases of this size, it can seem intimidating at first, so better to plan early.

2 Dev-kit

A dev-kit consisting of skeletons and other necessary files has been provided to you along with this document. You first need to clone the dev-kit to your home directory in the login node of the DSMLP cluster. When you spawn the cluster following the instructions in Section 6, this dev-kit should be prepared on your cluster's master node automatically, and you do not need to download it manually.

Within the dev-kit, there are several files:

```
assignment2.ipynb -- a playground for your development
assignment2.py -- the deliverable of this assignment, your final file to submit
-----
cluster-manager.sh --
spark-cluster.yaml.template --
pa2_main.py --
utilities.py -- above files are necessary for your code to run. Do not modify any of them.
```

3 Dataset Description

3.1 Schemas

For the first part you are expected to extract features from three tables, their schemas and descriptions are listed below:

1. product
 - |-- asin: string, the product id, e.g., 'B00I8HVV6E'
 - |-- salesRank: map, a map between category and sales rank, e.g., {'Home & Kitchen': 796318}
 - | |-- key: string, category, e.g., 'Home & Kitchen'
 - | |-- value: integer, rank, e.g., 796318
 - |-- categories: array, list of list of categories, e.g., [['Home & Kitchen', 'Artwork']]
 - | |-- element: array, list of categories, e.g., ['Home & Kitchen', 'Artwork']
 - | | |-- element: string, category, e.g., 'Home & Kitchen'

¹UCSD's Data Science and Machine Learning Platform

²datahub.ucsd.edu

```

|-- title: string, title of product, e.g., 'Intelligent Design Cotton Canvas'
|-- price: float, price of product, e.g., 27.9
|-- related: map, related information, e.g., {'also_viewed': ['B00I8HWOUK']}
|   |-- key: string, the attribute name of the information, e.g., 'also_viewed'
|   |-- value: array, array of product ids, e.g., ['B00I8HWOUK']
|   |   |-- element: string product id , e.g., 'B00I8HWOUK'
2. product_processed
|-- asin: string, same as above
|-- title: string, title column after imputation, e.g., 'Intelligent Design Cotton Canvas'
|-- category: string, category column after extraction, e.g., 'Home & Kitchen'
3. review
|-- asin: string, same as above
|-- reviewerID: string, the reviewer id, e.g., 'A1MIP8H7G33SHC'
|-- overall: float, the rating associated with the review, e.g., 5.0

```

The **review** table will be useful for extracting the rating information for each product in Task 1. We will be working primarily with **product** table throughout Task 1-4. **product_processed** is used for Task 5-6.

For the second part, you are expected to train ML models on the extracted features for predicting the user rating for a product. For your convenience, we have created ML-ready data that contain user and product features and also the rating. We will provide two tables, one for training and one for testing. The schemas of the two tables is shown below:

```

1. ml_features_train
|-- features: SparseVector(float), SparseVector of concatenated
features from user and product data (all features are continuous
features)
|-- overall: int, review rating
2. ml_features_test
|-- features: SparseVector(float), same as above
|-- overall: int, same as above

```

All the datasets required for this assignment can be found in a NFS directory and are correctly linked in the **utilities.py** file already. We will be reading from this NFS directly, which gets mounted with the Spark cluster. You do not need to download or modify any of them.

3.2 Sizes

Table 1 summarizes the sizes of the datasets on disk and in memory.

Table 1: Datasets sizes			
	Size/GiB (on disk)	Size/GiB (in memory)	Compressed (on disk)
product	5.6	10.1	False
product_processed	0.7	0.6	False
review	1.7	1.9	False
ml_features_train	3.4	10.2	True
ml_features_test	0.9	2.5	True
total	12.3	25.3	-

4 Tasks

Part 1 and 2 have 6 and 2 tasks, respectively. Altogether, you are required to complete eight tasks in total. In each task, you will need to implement a function **task_i()**. The function signatures and return types are fixed and provided to you in the dev-kit. Each function will take in several inputs and conduct the desired transformations. At the end of each task, you will be asked to extract several statistical properties (mean, variance, RMSE, etc.) from

the transformed data. You will need to put these properties in a python dictionary named `res` programmatically, the schema of the dictionary is also given. Each of the tasks will be tested in unit. It means each function you write will be tested in isolation from the rest. We would award partial points even if some tasks failed.

For the tasks, you can use any combinations of the Spark APIs available in the environment. However, you can only select (by setting a global variable called `INPUT_FORMAT`) one of the three APIs for inputs: `DataFrame`, `RDD`, `Koalas`. Inside your function body, you have the freedom to switch between them.

Important: Tasks 2, 3, 5, 6, 7, 8 **cannot** be solved solely with `Koalas`. Currently, `Koalas` does not support nested types (yet), so Task 2 and Task 3 are not doable with it. Also, it does not have the required ML support for Task 5, 6, 7, 8. You will need to switch to other APIs for these tasks.

4.0.1 Conventions

These rules apply to all the tasks.

Results format

Each task comes with a pre-defined schema for the output results. The result must be stored as a native python dictionary and must contain all the keys and nested structures. You must only use python's built-in datatypes. For instance, if your value is of datatype `np.float64`, you must first cast it into python `float`.

For the following schema:

```
res
| -- single_value: int -- an integer number
| -- list_of_values: list -- a list of values
| -- element: float -- a float number
```

A desired python code snippet to compose up the dictionary would be similar to:

```
1 data = ... # Your transformed data
2 res = {
3     'single_value': None,
4     'list_of_values': [None]
5 } # Skeleton given for the result
6 res['single_value'] = int(data.some_op())
7 res['list_of_values'] = [float(data.some_op()), float(data.some_op()), float(data.some_op())]
```

Handling null None and NaN

The input tables contain `null` (or `None` as in `RDD/python`, or `NaN` as in `Koalas/pandas`, we will be using these notations interchangeably) and dangling references. You do not need to deal with dangling reference unless instructed. For `null` values we will follow the common practice in SQL world: unless instructed otherwise, you need to ignore all `nulls` when calculating statistics such as count, mean and variance. Of course, do not ignore `null` when you are explicitly asked to count the number of `null` entries.

Column names for RDD users

Only relevant if you are using RDD API. We will be using column names a lot in the task descriptions. In `RDD` there are no named columns and you will need to convert the names to corresponding column indices yourself. Also you can ignore the directions about names of generated columns.

4.1 Task1: mean and count of ratings

First, you will aggregate and extract some information from the user review table. We want to know for each product, what are the mean and the number of ratings it received. Implement a function `task_1` that does the following:

1. For each product ID `asin` in `product` calculate the average rating it received in the column `meanRating`. The ratings are stored in column `overall` of `review`.
2. Similarly, put the count of ratings for each product in a new column named `countRating`.
3. You need to conduct the above operations, then extract some statistics out of the generated columns. You need to put the statistics in a python dictionary named `res`. The description and schema of it are as follows:

```
res
| -- count_total: int -- count of rows of the transformed table, including null rows
```

```
| -- mean_meanRating: float -- mean value of meanRating
| -- variance_meanRating: float -- variance of meanRating
| -- numNulls_meanRating: int -- count of nulls of meanRating
| -- mean_countRating: float -- mean value of countRating
| -- variance_countRating: float -- variance of countRating
| -- numNulls_countRating: int -- count of nulls of countRating
```

If for a product ID, there is not a single reference in `review` meaning it was never reviewed, you should put `null` in both `meanRating` and `countRating`.

4.2 Task 2: flatten categories and salesRank

Implement a function `task_2` to conduct the following operations:

1. For the `product` table, each item in column `categories` contains an array of arrays of hierarchical categories. The schema is `ArrayType(ArrayType(StringType))`. We are only going to use the most general category, which is the first element of the nested array: `array[0][0]`. For each row, put the first element of `categories` in a new column `category`. If `categories` is null or empty (e.g., put a `\verbnull`— in your new column.
2. On the other hand, each entry in column `salesRank` is a key-value pair: `bestSalesCategory, rank`). Your task is to flatten it into two columns. Put the key in a new column named `bestSalesCategory` and the value in `bestSalesRank`. Put null if the original entry was null or empty.
3. The schema of output is as follows:

```
res
| -- count_total: int -- count of rows of the transformed table, including null rows
| -- mean_bestSalesRank: float -- mean value of bestSalesRank
| -- variance_bestSalesRank: float -- variance of bestSalesRank
| -- numNulls_category: int -- count of nulls of category
| -- countDistinct_category: int -- count of distinct values of category, excluding nulls
| -- numNulls_bestSalesCategory: int -- count of nulls of bestSalesCategory
| -- countDistinct_bestSalesCategory: int -- count of distinct values of bestSalesCategory,
|                                     excluding nulls
```

4.3 Task 3: flatten related

Each entry of `related` column is a map with four keys/attributes: `also_bought`, `also_viewed`, `bought_together`, and `buy_after_viewing`. Each value of these keys contains an array of product IDs. We call them attribute arrays. You need to calculate the length of the arrays and find out the average prices of the products in these arrays.

The logic for all four attributes is identical. For the sake of simplicity, you are only required to flatten the `also_viewed` attribute. Your task is to implement function `task_3()` that does the following:

1. For each row of `related`: calculate the mean price of all products from the `also_viewed` attribute array. Put it in a new column `meanPriceAlsoViewed`. Remember to ignore the product IDs if they do not match any record in `product`. Or if they match records in `product`, but the records have `null` in the price column. Do **not** ignore products if they have `price = 0`.
2. Similarly, put the length of that array in a new column `countAlsoViewed`. In this case, you do **not** need to check if the product IDs in that array are dangling references and do not have matching records in `product`. Put `null` (instead of zero) in the new column, if the attribute array is `null` or empty.
3. The schema of output is as follows:

```
res
| -- count_total: int -- count of rows of the transformed table, including null rows
| -- mean_meanPriceAlsoViewed: float -- mean value of meanPriceAlsoViewed
| -- variance_meanPriceAlsoViewed: float -- variance of meanPriceAlsoViewed
| -- numNulls_meanPriceAlsoViewed: int -- count of nulls of meanPriceAlsoViewed
| -- mean_countAlsoViewed: float -- mean value of countAlsoViewed
| -- variance_countAlsoViewed: float -- variance of countAlsoViewed
| -- numNulls_countAlsoViewed: int -- count of nulls of countAlsoViewed
```

4.4 Task 4: data imputation

You may have noticed that there are lots of **nulls** in the table. Now your task is to impute them with meaningful values that can be used to train machine learning models.

Since the schema is already flattened, now we only have two data types in our table: numerical (including integer and floating numbers) and string. Now you need to impute a numerical column **price**, as well as a string column **title**. Please implement a function **task_4()**:

1. For column **price** first cast every entry to float type. Then impute the **nulls** with the mean of all the non-null values. Store the outputs in a new column **meanImputedPrice**.
2. Same as above, but this time impute with the **median** value. Store the imputed data in a new column **medianImputedPrice**.
3. For column **title**: As for the string-typed columns, we want to impute **nulls and empty strings** simply with a special string **unknown**'. Store the imputed data in a new column **unknownImputedTitle**.
4. The schema of output is as follows:

```
res
| -- count_total: int -- count of rows of the transformed table, including null rows
| -- mean_meanImputedPrice: float or None -- mean value of meanImputedPrice
| -- variance_meanImputedPrice: float -- variance of meanImputedPrice
| -- numNulls_meanImputedPrice: int -- count of nulls of meanImputedPrice
| -- mean_medianImputedPrice: float or None -- mean value of medianImputedPrice
| -- variance_medianImputedPrice: float -- variance of medianImputedPrice
| -- numNulls_medianImputedPrice: int -- count of nulls of medianImputedPrice
| -- numUnknowns_unknownImputedTitle: float -- count of 'unknown' value
entries in unknownImputedTitle
```

4.5 Task 5: embed title with word2vec

*This task assumes the title column is already imputed with **unknown**. We have provided the imputed data table **product_processed**.*

In this task, we want to transform **title** into a fixed-length vector via **word2vec**. You need to implement function **task_5()**:

1. For each row, convert **title** to lowercase, then split it by whitespace (' ') to an array of strings, store this array in a new column **titleArray**.
2. Train a **word2vec** model out of column **titleArray**. Do not try to implement **word2vec** yourself. Instead, use **M.feature.Word2Vec**. See the instructions below.
3. For each of the three words inputted as **<word_0>**, **<word_1>**, and **<word_2>**, use your obtained **word2vec** model to get the 10 closest synonyms along with similarity scores (cosine similarity of word vectors). **M.feature.Word2Vec** also has a built-in method for this task.
4. We will generate 100 synonyms for each of the three words with our solution code. For each word, if at least one of your synonyms matches ours, it is counted as pass for that word.
5. The schema of output is as follows:

```
res
| -- count_total: int -- count of rows of the transformed table, including null rows
| -- size_vocabulary: int -- the size of the vocabulary of your word2vec model
| -- word_0_synonyms: list -- synonyms tuples of word_0
| | -- element: tuple -- tuple of format (synonym, score)
| | | -- element: string -- synonym
| | | -- element: float -- score
| -- word_1_synonyms: list
| | -- element: tuple
| | | -- element: string
| | | -- element: float
| -- word_2_synonyms: list
| | -- element: tuple
| | | -- element: string
| | | -- element: float
```

M.feature.word2vec instructions:

1. Set `minCount`, the minimum number of times a token must appear to be included in the word2vec model's vocabulary, to 100.
2. Set the dimension of output word embedding to 16.
3. You need to set the random seed as `SEED`; this is a global variable defined to be 102.
4. Set `numPartitions` to 4.
5. You should keep all other settings as default.
6. `M.feature.Word2Vec` is not fully reproducible (although we have set the seed here). We are aware of the issue, and your score will not be affected by its internal randomness.

4.6 Task 6: one-hot encoding category and PCA

Assume `categories` is already flattened and `unknown` imputed for the input data. We have provided you with the preprocessed table.

Now you need to one-hot-encode the categorical features. Meanwhile, the categories may be correlated. So as a practice, we would like to run PCA on these categories. Implement function `task_6()`:

1. First one-hot encode `category` and put the resulted vectors in a new column `categoryOneHot`. Ensure the dimension of generated vectors equals to the size of domain. For example, if we have three categories in total: `V = {'Electronics', 'Books', 'Appliances'}`. Then the encoding for 'Electronics' can be `[1, 0, 0]` or `[0, 1, 0]` or `[0, 0, 1]`, but the dimension of this vector must be 3.

Hint: For `DataFrame`, before encoding a string-typed column, you may have to first convert it to a column of numerical indices with `M.feature.StringIndexer`. Then use `M.feature.OneHotEncoder` to do the encoding. Set `dropLast` argument to false.

For `RDD`, you may need to implement the one-hot-encoding logic yourself. Consider building the one-hot mapping locally, then broadcasting and map it to every row.

2. Apply PCA on the one-hot-encoded column. Reduce the dimension of each one-hot vector to 15, put the transformed vectors in a new column `categoryPCA`. On `DataFrame`, use `M.feature.PCA`. On `RDD`, see instructions.

3. Column `categoryOneHot` and `categoryPCA` will be of `VectorType` (in `DataFrame`) or python iterable type (in `RDD`). You do not need to worry if the vectors are sparsely or densely represented. However, in the results you will be asked to calculate the mean vectors, for these we only accept dense representation.

4. The schema of output is as follows::

```
res
| -- count_total: int -- count of rows of the transformed table, including null rows
| -- meanVector_categoryOneHot: list -- mean vector of transformed one-hot-encoding vectors,
densely represented
| | -- element: float -- element of the mean vector, from first to last dimension
| -- meanVector_categoryPCA: list -- mean vector of the PCA-transformed vectors,
densely represented
| | -- element: float
```

Mean vector: If you have three vectors `[1, 0, 0]`, `[0, 1, 0]`, and `[0, 1, 0]`, then the mean vector is `[1/3, 2/3, 0]`, i.e., taking the mean on each component/direction. To calculate mean vector, consider using `ML.stat.Summarizer`.

4.7 Task 7: Train a Decision Tree Regression model

We are providing you with two processed ML-ready feature tables for training and testing ML models. Each record corresponds to a review and contains features from the corresponding product (e.g., price, average product rating, and average also bought count etc.) and the user (e.g., number of products rated, and average user rating etc.). Materializing this joint feature table by joining features from product and user tables takes significant amount of time (over 30 minutes) and hence we provide you with pre-materialized data.

Now you need to train a Decision Tree Regression model using this data to predict the user rating for a product.

1. Implement function `task_7()`. Train a Decision Tree Regression model using the training data. The **max tree depth** parameter of the model **must be set to 5**. All other parameters of the model should be left to default values.

2. Use the trained model to generate predictions on test data. Calculate the root mean square error (RMSE) of the test predictions and report it in the output.
3. The schema of output is as follows:

```
res
| -- test_rmse: float -- RMSE of the test predictions
```

4.8 Task 8: Hyperparameter tuning for the Decision Tree Regression model

In `task_7()`, we fixed the max tree depth parameter of the model and trained a single model. Now we perform hyperparameter tuning to select the best max tree depth.

1. First, create new training and validation data from the original training data. Use a random split of 75/25.
2. Train Decision Tree Regression models with max tree depth values of 5, 7, 9, and 12. Also, calculate the RMSE of validation data predictions and report it in the output.
3. Based on the validation RMSE values, pick the best model, and use it to generate predictions on test data. Report test RMSE in the output.
4. The schema of output is as follows::

```
res
| -- test_rmse: float -- RMSE of the test predictions generated by the best model based on
validation RMSEs
| -- valid_rmse_depth_5: float -- RMSE of the validation set predictions
generate by max tree depth of 5
| -- valid_rmse_depth_7: float -- same as above but w/ depth 7
| -- valid_rmse_depth_9: float -- same as above but w/ depth 9
| -- valid_rmse_depth_12: float -- same as above but w/ depth 12
```

5 Deliverables

Code up all the tasks in the designated places in `assignment2.py`. Then rename the file to `assignment2_<your team id>.py`. For instance, if your team id is 18, then your filename would be `assignment2.18.py`. Use SCP or Jupyter Notebook or any other tools to download the `assignment2.py` file to your own machine. Submit this file on Canvas; only one team member needs to do so.

6 Getting started

6.1 How to read and copy commands in this section

1. In this section, we have three different hosts where you can type commands: your computer (`local`), the login node (`dsmlp-login`), and Spark master node (`spark-master`). All shell commands will be given you in the format of:

```
@<host>: <commands>
```

For instance, if we would like you to list the directory on your computer, the command would be:

```
@local: ls
```

In this scenario, what you need to do is open a terminal (Linux and OS X users) or a PowerShell (Windows users), copy-paste `ls`, and execute it.

2. On the other hand, if you are given a command like:

```
@dsmlp-login: ls
```

This means the command `ls` needs to be executed on the login node. You need to first SSH into it and then execute the command. We will show you how to do the SSH.

3. Sometimes you may encounter angular brackets `<XXX>`; in this situation, you will need to substitute it with the desired value. Do **not** leave the brackets. For example, the following command

```
@local: echo <pid>
```

You need to put your pid in the command, and the command you run would become (assuming your pid is a10000000):

```
@local: echo a10000000
```

4. Be extra cautious when copy-pasting commands. Quite often copying from pdf will lead to missing/extra characters. If anything does not run, please first ensure your pasted command is exactly the same as listed in this document.

6.2 Prepare the Dev-Kit via Datahub (Updated Method)

You no longer need to SSH into the login node to copy the assignment files. Instead, use the terminal in Datahub to set up your development environment.

1. Go to <https://datahub.ucsd.edu/hub/login>.
2. Click the large yellow **Log In** button as in Figure 2.
3. In the launch interface, select the **DSC-102** tab, then click **Launch Environment** (see Figure 2).
4. Once the JupyterLab interface loads, click **New** in the top-right corner, then select **Terminal** (see Figure 3).
5. In the terminal, run the following command to copy the starter files:

```
bash-5.1: cp -r ./public/DSC102-PA2 ./private/
```

This will create a folder named “DSC102-PA2” in your private directory. You will work in this directory for the assignment.

The working directory of this Jupyter notebook is the home directory of your login node. So all your modifications to the assignment files will be saved, and no files are stored in the cluster. In Jupyter Notebook, go to directory DSC102-PA2/src, rename `assignment2.ipynb` to `assignment2_<your pid>.ipynb` and continue the assignment there.

You can now begin the assignment by editing your renamed notebook in the Jupyter interface.

6.3 (Optional) SSH Method for Advanced Users

Alternatively, if you’re comfortable using SSH, you can still use the login node:

1. Open a terminal on your local machine.
2. Use your ETS account and password to SSH into the login node:

```
@local: ssh <ETS account>@dsmlp-login.ucsd.edu
```

3. If you want to mimic the Datahub environment, run:

```
@dsmlp-login: launch-spark.sh -W DSC102_SP25_A00
```

4. Then copy the starter files:

```
@dsmlp-login: cp -r ./public/DSC102-PA2 ./private/
```

Your ETS account name is usually the same as your UCSD email name. If you have trouble finding it or you forgot the password, use ETS Account Lookup.



Figure 1: Navigating to Datahub.

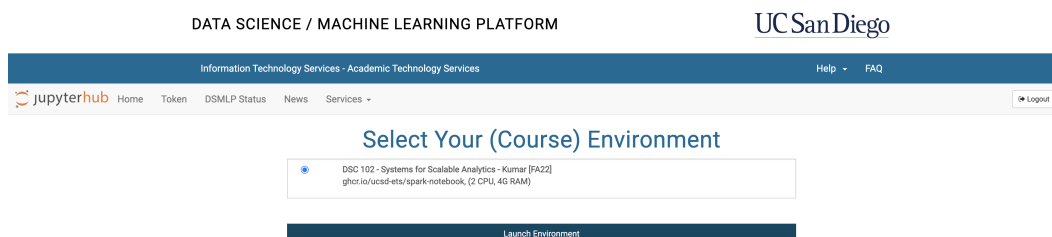


Figure 2: Launch the environment in Datahub.

6.4 Where to go

Head to <https://datahub.ucsd.edu/hub/login> as shown in Figure 1, click the large yellow Log In button on the right hand side. From there you should see a launch environment tab that, click the DSC-102 tab above the Launch Environment button, and then click the Launch Environment button, as shown in Figure 2. Doing this initializes a spark cluster for you to use. Then head on over to a new terminal on the homepage, find New in the top right hand corner, then click on Terminal as shown in Figure 3. In the terminal we want to run:

```
bash-5.1: kubectl get pods
```

This is shown in Figure 4, note that before you develop in your Jupyter notebook it is important for all of the workers to initialize first. The top half of Figure 4 shows a worker not ready to be used (the ready column has a 0/1), the bottom half should be what you see before you start developing. You may have to run the command a few times before you see the worker initialized. From there headback to the home page, and the DSC102-PA2 folder should be in the private directory. Click on the private directory, and then head to the DSC102-PA2 folder, then `src`.

6.5 Develop your solutions in Jupyter Notebook

You will first develop, test, and debug your solutions in Jupyter Notebook.

Develop. In the notebook (as shown in Figure 5) it runs PySpark and you have a live session with the Spark cluster, just like you had with Dask in PA1. Code up your solutions in the function skeletons provided to you.

Test. Immediately after each function skeleton, we provide you with code to run your implementation and subsequently test it. Your outputs are returned immediately.

The desired results, generated by our solutions, are provided to you along side with the dataset. You can find them from your login node's shell:

```
@dsmlp-login: cd ./<your pid>-pa2/test_results
```

You can find our results in json files with names of format `"task_i.json"`.

Alternatively, you can also view these results in your Jupyter Notebook by inspecting a dictionary named `"pa2.tests.res_dict"`, the keys to this dict are of format `"task_i"`.

Debug. The tests will notify you if any results did not match. If you are experiencing crashing/slowness, you can monitor the health situation of the cluster in the Spark cluster manager UI (screen shot Figure 6). This can

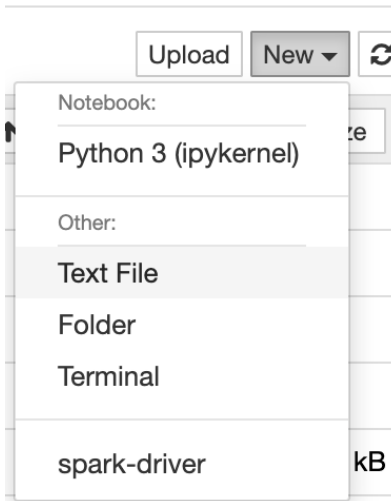


Figure 3: Find the New button then click on terminal.

```
bash-5.1$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
dsmlp-jupyter-tjtuttle 1/1     Running   0           90s
spark-master-0        1/1     Running   0           84s
spark-worker-0         1/1     Running   0           85s
spark-worker-1         0/1     Running   0           42s
bash-5.1$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
dsmlp-jupyter-tjtuttle 1/1     Running   0           2m12s
spark-master-0        1/1     Running   0           2m6s
spark-worker-0         1/1     Running   0           2m7s
spark-worker-1         1/1     Running   0           84s
bash-5.1$
```

Figure 4: Example of the kubectl get pods command.

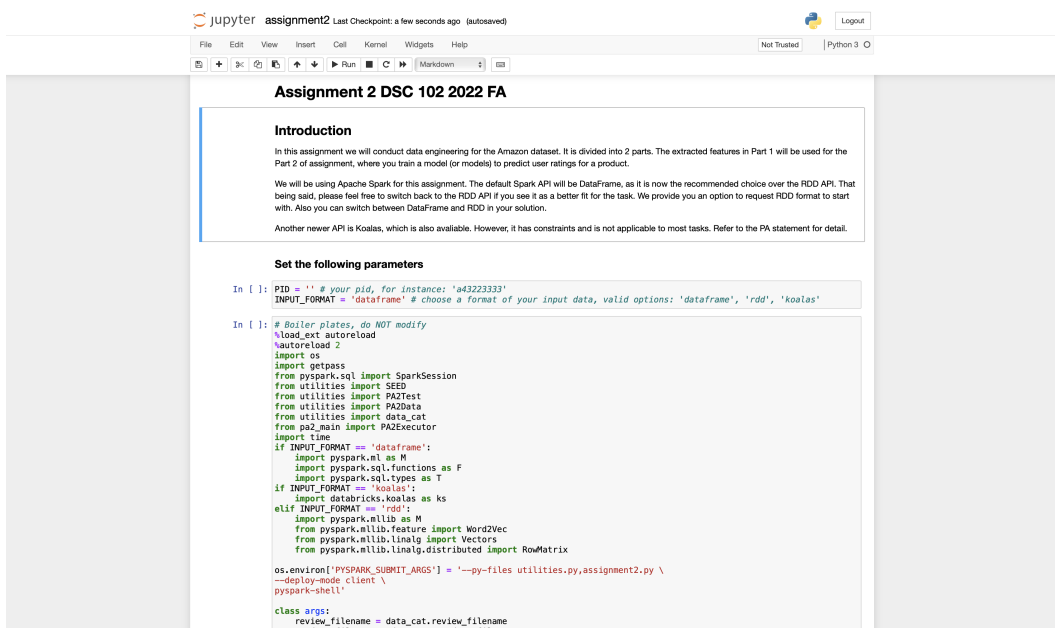
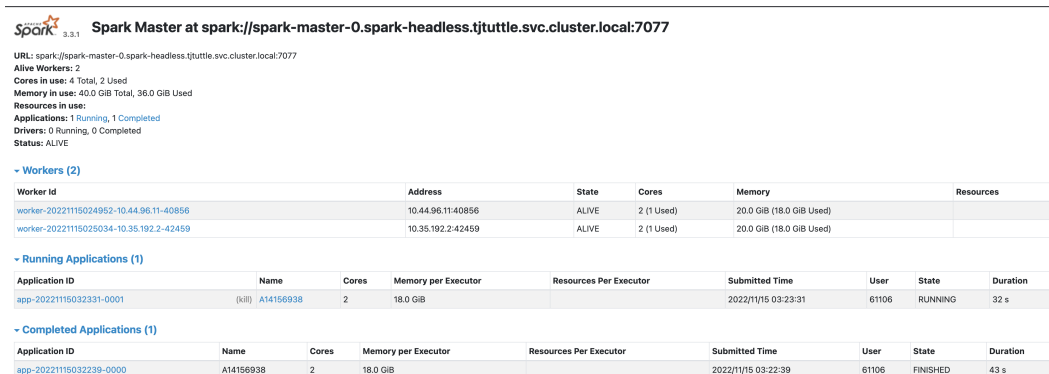


Figure 5: Developing, testing, and debugging in Jupyter Notebook.



Spark Master at spark://spark-master-0.spark-headless.tjuttile.svc.cluster.local:7077

URL: spark://spark-master-0.spark-headless.tjuttile.svc.cluster.local:7077

Alive Workers: 2

Cores in use: 4 Total, 2 Used

Memory in use: 40.0 GiB Total, 36.0 GiB Used

Resources in use:

Applications: 1 Running, 1 Completed

Drivers: 0 Running, 0 Completed

Status: ALIVE

Workers (2)

Worker Id	Address	State	Cores	Memory	Resources
worker-2022115024952-10.44.96.11-40856	10.44.96.11-40856	ALIVE	2 (1 Used)	20.0 GiB (18.0 GiB Used)	
worker-2022115025034-10.35.192.2-42459	10.35.192.2-42459	ALIVE	2 (1 Used)	20.0 GiB (18.0 GiB Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-2022115032331-0001	(k8s) A14156938	2	18.0 GiB		2022/11/15 03:23:31	61106	RUNNING	32 s

Completed Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-2022115032239-0000	A14156938	2	18.0 GiB		2022/11/15 03:22:39	61106	FINISHED	43 s

Figure 6: Spark cluster manager UI

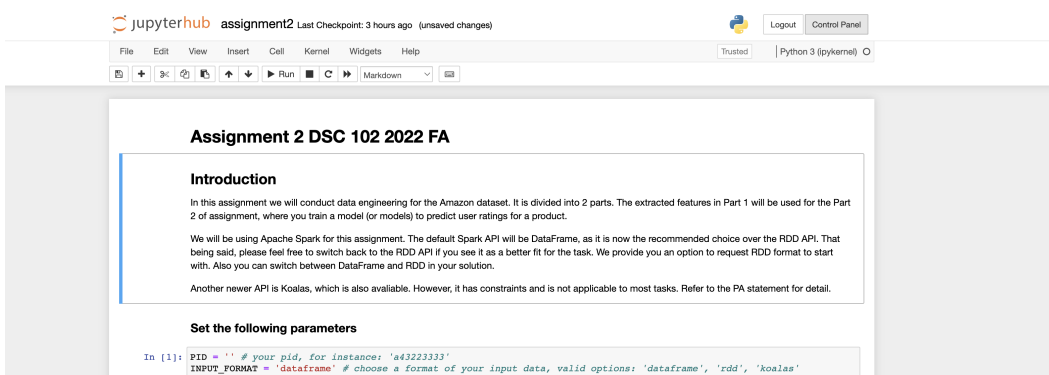


Figure 7: Where to find the Control Panel.

be found by going back to the homepage and clicking New, then clicking spark-driver at the bottom as shown in Figure 3.

6.6 Restarting your cluster

Simply logging out of datahub does not stop the spark cluster. If you need to stop the spark cluster click the Control Panel in the top right hand corner (Figure 7) , and then click the red “Stop My Server” button as shown in Figure 8. After this click the “Start My Server” button, which should prompt you back to the launch environment from Figure 2.

6.7 Final test of your solution before turning in

You will **not** turn in the notebook. Instead, you need to put your implementations of **task_1** to **task_8**, along with all the dependencies you imported and helper functions you defined, in the file co-located with the notebook: **assignment2.py**.

If you are collaborating in a team, at this stage, please combine your work into one single file. Only **one** person needs to turn in the final file. Do **not** modify the filename before you upload it to Canvas.

Before turning in the file, you need to make sure your script runs with the given commands, or you may lose points. Our auto-grader will take your python file and runs it with **spark-submit**, so it is vital to make sure your code can execute without issues this way.

1. From the shell on the login node, query the master node’s pod name:

```
@dsmlp-login: kubectl get pods
```

The name would be in the format of “**spark-master-XXX-XXX**”.

2. SSH into the master node via

Stop My Server
My Server

Figure 8: Stopping your server stops your spark cluster.

```
@dsmlp-login: kubectl exec -it <spark-master-XXX-XXX> -- bash
```

3. In the Spark master node shell, go to your root directory of scripts

```
@spark-master: cd /home/<your ETS Account name>/private/DSC102-PA2/src
```

4. If your Jupyter Notebook is still running, shutdown it in Link to free up the workers.

5. Run PA2 with the following command, do not modify anything except “<your pid>” and “<your ETS account name>”:

```
@spark-master: spark-submit \
--py-files utilities.py,assignment2.py \
--files log4j-spark.properties \
--deploy-mode client \
--driver-java-options "-Dlog4j.configuration=file:log4j-spark.properties" \
--master spark://spark-master-svc:7077 \
--conf "spark.executor.extraJavaOptions=-Dlog4j.configuration=file:log4j-spark.properties" \
pa2_main.py --pid <your pid> --username <your ETS account name>
```

Make sure your script can execute and try to pass as many tests as you can.